

Fft

快速傅里叶变换

前置知识：[复数](#)。

本文将介绍一种算法，它支持在 $O(N \log N)$ 的时间内计算两个 N 次多项式的乘法，比朴素的 $O(N^2)$ 算法更高效。由于两个整数的乘法也可以被当作多项式乘法，因此这个算法也可以用来加速大整数的乘法计算。

引入

我们现在引入两个多项式 $A(x)$ 和 $B(x)$ ：

两个多项式相乘的积 $C(x)$ ，我们可以在 $O(N^2)$ 的时间复杂度中解得（这里 N 为 $A(x)$ 或者 $B(x)$ 多项式的次数）：

很明显，多项式 $C(x)$ 的系数 c_k 满足 $c_k = \sum_{i+j=k} a_i b_j$ 。而对于这种朴素算法而言，计算每一项的时间复杂度都为 $O(N)$ ，一共有 N 项，那么时间复杂度为 $O(N^2)$ 。

能否加速使得它的时间复杂度降低呢？如果使用快速傅里叶变换的话，那么我们可以使得其复杂度降低到 $O(N \log N)$ 。

傅里叶变换

傅里叶变换（Fourier Transform）是一种分析信号的方法，它可分析信号的成分，也可用这些成分合成信号。许多波形可作为信号的成分，傅里叶变换用正弦波作为信号的成分。

设 $f(t)$ 是关于时间 t 的函数，则傅里叶变换可以检测频率 ω 的周期在 $f(t)$ 出现的程度：

它的逆变换是

逆变换的形式与正变换非常类似，分母 2π 恰好是指数函数的周期。

傅里叶变换相当于将时域的函数与周期为 2π 的复指数函数进行连续的内积。逆变换仍旧为一个内积。

傅里叶变换有相应的卷积定理，可以将时域的卷积转化为频域的乘积，也可以将频域的卷积转化为时域的乘积。

离散傅里叶变换

离散傅里叶变换（Discrete Fourier transform, DFT）是傅里叶变换在时域和频域上都呈离散的形式，将信号的时域采样变换为其 DTFT（discrete-time Fourier transform）的频域采样。

傅里叶变换是积分形式的连续的函数内积，离散傅里叶变换是求和形式的内积。

设 $x[n]$ 是某一满足有限性条件的序列，它的离散傅里叶变换（DFT）为：

其中 e 是自然对数的底数， j 是虚数单位。通常以符号 ω_N 表示这一变换，即

类似于积分形式，它的 **逆离散傅里叶变换**（IDFT）为：

可以记为：

实际上，DFT 和 IDFT 变换式中和式前面的归一化系数并不重要。在上面的定义中，DFT 和 IDFT 前的系数分别为 $1/N$ 和 1 。有时我们会将这两个系数都改 $1/\sqrt{N}$ 。

离散傅里叶变换仍旧是时域到频域的变换。由于求和形式的特殊性，可以有其他的解释方法。

如果把序列 $x[n]$ 看作多项式 $X(z)$ 的 N 项系数，则计算得到的 $X(\omega_k)$ 恰好是多项式 $X(z)$ 代入单位根 $z = \omega_N^k$ 的点值。

这便构成了卷积定理的另一种解释办法，即对多项式进行特殊的求值操作。离散傅里叶变换恰好是多项式在单位根处进行求值。

例如计算：

定义函数 $X(z)$ 为：

然后可以发现，代入四次单位根 ω_4 得到这样的序列：

于是下面的求和恰好可以把其余各项消掉：

因此这道数学题的答案为：

这道数学题在单位根处求值，恰好构成离散傅里叶变换。

矩阵公式

由于离散傅立叶变换是一个 **线性** 算子，所以它可以用矩阵乘法来描述。在矩阵表示法中，离散傅立叶变换表示如下：

其中 F_N 。

快速傅里叶变换

FFT 是一种高效实现 DFT 的算法，称为快速傅立叶变换（Fast Fourier Transform，FFT）。它对傅里叶变换的理论并没有新的发现，但是对于在计算机系统或者说数字系统中应用离散傅立叶变换，可以说是进了一大步。快速数论变换（NTT）是快速傅里叶变换（FFT）在数论基础上的实现。

在 1965 年，Cooley 和 Tukey 发表了快速傅里叶变换算法。事实上 FFT 早在这之前就被发现过了，但是在当时现代计算机并未问世，人们没有意识到 FFT 的重要性。一些调查者认为 FFT 是由 Runge 和 König 在 1924 年发现的。但事

实上高斯早在 1805 年就发明了这个算法，但一直没有发表。

分治法实现

FFT 算法的基本思想是分治。就 DFT 来说，它分治地来求 ω_N^k 的值。基 - 2 FFT 的分治思想体现在将多项式分为奇次项和偶次项处理。

举个例子，对于一共 N 项的多项式：

按照次数的奇偶来分成两组，然后右边提出来一个：

分别用奇偶次项数建立新的函数：

那么原来的 $P(x)$ 用新函数表示为：

利用偶数次单位根的性质， ω_N^{2k} 和 $\omega_N^{2k+N/2}$ 是偶函数，我们知道在复平面上 ω_N^{2k} 和 $\omega_N^{2k+N/2}$ 对应的值相同。得到：

和：

因此我们求出了 P_1 和 P_2 后，就可以同时求出 P_0 和 P_1 。于是对 P_1 和 P_2 分别递归 DFT 即可。

考虑到分治 DFT 能处理的多项式长度只能是 2^k ，否则在分治的时候左右不一样长，右边就取不到系数了。所以要在第一次 DFT 之前就把序列向上补成长度为 2^k （高次系数补 0）、最高项次数为 $2^k - 1$ 的多项式。

在代入值的时候，因为要代入 2^k 个不同值，所以我们代入 2^k 个不同值。

代码实现方面，STL 提供了复数的模板，当然也可以手动实现。两者区别在于，使用 STL 的 `complex` 可以调用 `exp` 函数求出 ω_N^k 。但事实上使用欧拉公式得到的虚数来求 ω_N^k 也是等价的。

以上就是 FFT 算法中 DFT 的介绍，它将一个多项式从系数表示法变成了点值表示法。

值得注意的是，因为是单位复根，所以说我们需要令 x 项式的高位补为零，使得 P_1 和 P_2 的系数在 $0 \sim N/2 - 1$ 范围内。

▼ 递归版 FFT

```

1 #include <LTCmath>
2 #include <LTcomplex>
3
4 using Comp = std::complex<double>; // STL complex
5
6 constexpr Comp I(0, 1); // i
7 constexpr int MAX_N = 1 << 20;
8
9 Comp tmp[MAX_N];
10
11 // rev=1,DFT; rev=-1,IDFT
12 void DFT(Comp* f, int n, int rev) {
13     if (n == 1) return;
14     for (int i = 0; i < n; ++i) tmp[i] = f[i];
15     // 偶数放左边, 奇数放右边
16     for (int i = 0; i < n; ++i) {
17         if (i & 1)
18             f[n / 2 + i / 2] = tmp[i];
19         else
20             f[i / 2] = tmp[i];
21     }
22     Comp *g = f, *h = f + n / 2;
23     // 递归 DFT
24     DFT(g, n / 2, rev), DFT(h, n / 2, rev);
25     // cur 是当前单位复根, 对于 k = 0 而言, 它对应的单位复根  $\omega^{0_n} = 1$ 。
26     // step 是两个单位复根的差, 即满足  $\omega^{k_n} = \text{step} * \omega^{k-1_n}$ ,
27     // 定义等价于  $\exp(I * (2 * M\_PI / n * \text{rev}))$ 
28     Comp cur(1, 0), step(cos(2 * M_PI / n), sin(2 * M_PI * rev / n));
29     for (int k = 0; k < n / 2;
30         ++k) { //  $F(\omega^{k_n}) = G(\omega^{k_{n/2}}) + \omega^{k_n} * H(\omega^{k_{n/2}})$ 
31         tmp[k] = g[k] + cur * h[k];
32         //  $F(\omega^{k_{n/2} * n}) = G(\omega^{k_{n/2}}) - \omega^{k_n} * H(\omega^{k_{n/2}})$ 
33         tmp[k + n / 2] = g[k] - cur * h[k];
34         cur *= step;
35     }
36     for (int i = 0; i < n; ++i) f[i] = tmp[i];
37 }

```

时间复杂度。

倍增法实现

这个算法还可以从「分治」的角度继续优化。对于基 - 2 FFT，我们每一次都会把整个多项式的奇数次项和偶数次项系数分开，一直分到只剩下一个系数。但是，这个递归的过程需要更多的内存。因此，我们可以先「模仿递归」把这些系数在原数组中「拆分」，然后再「倍增」地去合并这些算出来的值。

对于「拆分」，可以使用位逆序置换实现。

对于「合并」，使用蝶形运算优化可以做到只用 的额外空间来完成。

位逆序置换

以 项多项式为例，模拟拆分的过程：

- 初始序列为
- 一次二分之一之后
- 两次二分之一之后
- 三次二分之一之后

规律：其实就是原来的那个序列，每个数用二进制表示，然后把二进制翻转对称一下，就是最终那个位置的下标。比如

是 001，翻转是 100，也就是 4，而且最后那个位置确实是 4。我们称这个变换为位逆序置换（bit-reversal permutation），证明留给读者自证。

根据它的定义，我们可以在 $O(N \log N)$ 的时间内求出每个数变换后的结果：

▼ 位逆序置换实现 (C)

```
1 /*
2  * 进行 FFT 和 IFFT 前的反置变换
3  * 位置 i 和 i 的二进制反转后的位置互换
4  * len 必须为 2 的幂
5  */
6 void change(Complex y[], int len) {
7     // 一开始 i 是 0...01, 而 j 是 10...0, 在二进制下相反对称。
8     // 之后 i 逐渐加一, 而 j 依然维持着和 i 相反对称, 一直到 i = 1...11。
9     for (int i = 1, j = len / 2, k; i < len - 1; i++) {
10         // 交换互为小标反转的元素, i < j 保证交换一次
11         if (i < j) swap(y[i], y[j]);
12         // i 做正常的 + 1, j 做反转类型的 + 1, 始终保持 i 和 j 是反转的。
13         // 这里 k 代表了 0 出现的最高位。j 先减去高位的全为 1 的数字, 直到遇到了
14         // 0, 之后再加上即可。
15         k = len / 2;
16         while (j >= k) {
17             j = j - k;
18             k = k / 2;
19         }
20         if (j < k) j += k;
21     }
22 }
```

实际上，位逆序置换可以 从小到大递推实现，设 N 表示二进制数的长度，设 $f(N)$ 表示长度为 N 的二进制数 翻转后的数（高位补 0）。我们要求的是 $f(N)$ 。

首先。

我们从小到大求 $f(N)$ 。因此在求 $f(N)$ 时， $f(N/2)$ 的值是已知的。因此我们把 $f(N/2)$ 右移一位（除以 2），然后翻转，再右移一位，就得到了 $f(N)$ 除了（二进制）个位 之外其它位的翻转结果。

考虑个位的翻转结果：如果个位是 0，翻转之后最高位就是 0。如果个位是 1，则翻转后最高位是 1，因此还要加上 $2^{N/2}$ 。综上

举个例子：设 $N=4$ ， $f(4)$ 。为了翻转：

1. 考虑 $f(2)$ ，我们知道 $f(2)=1$ ，再右移一位就得到了 $f(4)/2$ 。
2. 考虑个位，如果是 1，它就要翻转到数的最高位，即翻转数加上 $2^{N/2}$ ，如果是 0 则不用更改。

▼ 位逆序置换实现 (C)

```

1 // 同样需要保证 len 是 2 的幂
2 // 记 rev[i] 为 i 翻转后的值
3 void change(Complex y[], int len) {
4     for (int i = 0; i < len; ++i) {
5         rev[i] = rev[i >> 1] >> 1;
6         if (i & 1) { // 如果最后一位是 1, 则翻转成 len/2
7             rev[i] |= len >> 1;
8         }
9     }
10    for (int i = 0; i < len; ++i) {
11        if (i < rev[i]) { // 保证每对数只翻转一次
12            swap(y[i], y[rev[i]]);
13        }
14    }
15    return;
16 }

```

蝶形运算优化

已知 x 和 y 后, 需要使用下面两个式子求出 u 和 v :

使用位逆序置换后, 对于给定的 :

- 的值存储在数组下标为 i 的位置, 的值存储在数组下标为 $rev[i]$ 的位置。
- 的值将存储在数组下标为 $rev[i]$ 的位置, 的值将存储在数组下标为 i 的位置。

因此可以直接在数组下标为 i 和 $rev[i]$ 的位置进行覆写, 而不用开额外的数组保存值。此方法即称为 **蝶形运算**, 或更准确的, 基 - 2 蝶形运算。

再详细说明一下如何借助蝶形运算完成所有段长度为 N 的合并操作:

1. 令段长度为 N ;
2. 同时枚举序列 x 的左端点 i 和序列 y 的左端点 j ;
3. 合并两个段时, 枚举 k , 此时 $x[k]$ 存储在数组下标为 $i+k$ 的位置, $y[k]$ 存储在数组下标为 $j+k$ 的位置;
4. 使用蝶形运算求出 u 和 v , 然后直接在原位置覆写。

快速傅里叶逆变换

傅里叶逆变换可以用傅里叶变换表示。对此我们有两种理解方式。

线性代数角度

IDFT (傅里叶反变换) 的作用, 是把目标多项式的点值形式转换成系数形式。而 DFT 本身是个线性变换, 可以理解为将目标多项式当作向量, 左乘一个矩阵得到变换后的向量, 以模拟把单位复根代入多项式的过程:

现在已经得到最左边的结果了, 中间的 x 值在目标多项式的点值表示中也是一一对应的, 所以, 根据矩阵的基础知识, 我们只要在式子两边左乘中间那个大矩阵的逆矩阵就行了。

由于这个矩阵的元素非常特殊, 它的逆矩阵也有特殊的性质, 就是每一项 **取倒数**, 再 **除以变换的长度**, 就能得到它的逆矩阵。

注意: 傅里叶变换的长度, 并不是多项式的长度, 变换的长度应比乘积多项式的长度长。待相乘的多项式不够长, 需要在高次项处补 0。

为了使计算的结果为原来的倒数，根据欧拉公式，可以得到

因此我们可以尝试着把单位根 取成，这样我们的计算结果就会变成原来的倒数，之后唯一多的操作就只有再除以它的长度，而其它的操作过程与 DFT 是完全相同的。我们可以定义一个函数，在里面加一个参数 或者是，然后把它乘到 上。传入 就是 DFT，传入 就是 IDFT。

单位复根周期性

利用单位复根的周期性同样可以理解 IDFT 与 DFT 之间的关系。

考虑原本的多项式是。而 IDFT 就是把你的点值表示还原为系数表示。

考虑 构造法。我们已知，求。构造多项式如下

相当于把 当做多项式 的系数表示法。

这时我们有两种推导方式，这对应了两种实现方法。

方法一

设，则多项式 在 处的点值表示法为。

对 的定义式做一下变换，可以将 表示为

记。

当 时，。

当 时，我们错位相减

也就是说

那么代回原式

也就是说给定点，则 的点值表示法为

综上所述，我们取单位根为其倒数，对 跑一遍 FFT，然后除以 即可得到 的系数表示。

方法二

我们直接将 代入。

推导的过程与方法一大同小异，最终我们得到。

当且仅当 时有，否则为。因此。

这意味着我们将 做 DFT 变换后除以，再反转后 个元素，同样可以还原 的系数表示。

代码实现

所以我们 FFT 函数可以集 DFT 和 IDFT 于一身。代码实现如下：

▼ 非递归版 FFT（对应方法一）

```
1 /*
2  * 做 FFT
3  * len 必须是 2^k 形式
4  * on == 1 时是 DFT, on == -1 时是 IDFT
5  */
6 void fft(Complex y[], int len, int on) {
7     // 位逆序置换
8     change(y, len);
9     // 模拟合并过程，一开始，从长度为一合并到长度为二，一直合并到长度为 len。
10    for (int h = 2; h <= len; h <= 1) {
11        // wn: 当前单位复根的间隔: w^1_h
12        Complex wn(cos(2 * PI / h), sin(on * 2 * PI / h));
13        // 合并, 共 len / h 次。
14        for (int j = 0; j < len; j += h) {
15            // 计算当前单位复根，一开始是 1 = w^0_n, 之后是以 wn 为间隔递增: w^1_n
16            // ...
17            Complex w(1, 0);
18            for (int k = j; k < j + h / 2; k++) {
19                // 左侧部分和右侧是子问题的解
20                Complex u = y[k];
21                Complex t = w * y[k + h / 2];
22                // 这就是把两部分分治的结果加起来
23                y[k] = u + t;
24                y[k + h / 2] = u - t;
25                // 后半部 「step」 中的 $\omega$ 一定和 「前半部」 中的成相反数
26                // 「红圈」上的点转一整圈「转回来」，转半圈正好转成相反数
27                // 一个数相反数的平方与这个数自身的平方相等
28                w = w * wn;
29            }
30        }
31    }
32    // 如果是 IDFT，它的逆矩阵的每一个元素不只是原元素取倒数，还要除以长度 len。
33    if (on == -1) {
34        for (int i = 0; i < len; i++) {
35            y[i].x /= len;
36            y[i].y /= len;
37        }
38    }
39 }
```

▼ 非递归版 FFT（对应方法二）

```

1 /*
2  * 做 FFT
3  * len 必须是 2^k 形式
4  * on == 1 时是 DFT, on == -1 时是 IDFT
5  */
6 void fft(Complex y[], int len, int on) {
7     change(y, len);
8     for (int h = 2; h <= len; h <= 1) {          // 模拟合并过程
9         Complex wn(cos(2 * PI / h), sin(2 * PI / h)); // 计算当前单位复根
10        for (int j = 0; j < len; j += h) {
11            Complex w(1, 0); // 计算当前单位复根
12            for (int k = j; k < j + h / 2; k++) {
13                Complex u = y[k];
14                Complex t = w * y[k + h / 2];
15                y[k] = u + t; // 这就是把两部分分治的结果加起来
16                y[k + h / 2] = u - t;
17                // 后半部 「step」 中的 $\omega$ 一定和 「前半部」 中的成相反数
18                // 「红圈」上的点转一整圈「转回来」, 转半圈正好转成相反数
19                // 一个数相反数的平方与这个数自身的平方相等
20                w = w * wn;
21            }
22        }
23    }
24    if (on == -1) {
25        reverse(y + 1, y + len);
26        for (int i = 0; i < len; i++) {
27            y[i].x /= len;
28            y[i].y /= len;
29        }
30    }
31 }

```

► FFT 模板 ([HDU 1402 - A * B Problem Plus](#))

参考文献

1. [桃酱的算法笔记](#).

本页面最近更新: 2024/11/27 23:32:04, [更新历史](#)

发现错误? 想一起完善? [在 GitHub 上编辑此页!](#)

本页面贡献者: [H-J-Granger](#), [ranwen](#), [abc1763613206](#), [Ahacad](#), [Allenyou1126](#), [AndrewWayne](#), [AngelKitty](#), [AtomAlpaca](#), [Back11ght](#), [billchenchina](#), [c-forrest](#), [CCXXXI](#), [Cheuring](#), [Chrogeek](#), [ChungZH](#), [countercurrent-time](#), [DepletedPrism](#), [Early0v0](#), [EarthMessenger](#), [Enter-tainer](#), [F1shAndCat](#), [GavinZhengOI](#), [Gesrua](#), [Great-designer](#), [greyqz](#), [Haohu Shen](#), [henryrabbit](#), [heroming](#), [hly1204](#), [lr1d](#), [isdanni](#), [jiang1997](#), [kenlig](#), [Lewy Zeng](#), [lucifer1004](#), [Menci](#), [muoshuoshu](#), [NachtgeistW](#), [needtoalmdown](#), [opsiff](#), [ouuan](#), [ouuan](#), [partychicken](#), [schtonn](#), [Sshwy](#), [sshwy](#), [StudyingFather](#), [SukkaW](#), [Tiphereth-A](#), [TrisolarisHD](#), [untitledunrevised](#), [Xeonacid](#), [YouXam](#), [Yukimaikoriya](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供, 附加条款亦可能应用